The Three Cups Game

Birmingham, 9th April 2024

Marcin Zelewski, MSc. Eng. Computer Science

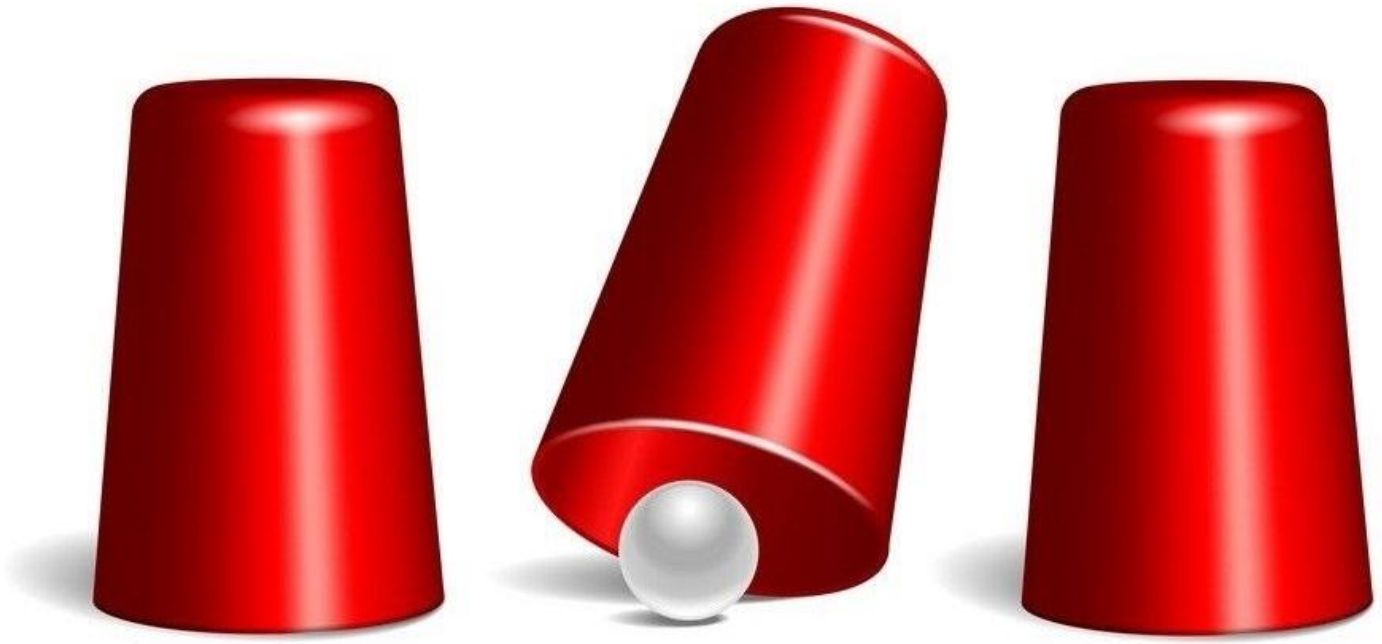
Experience and Expertise

- Freelance Software Developer, 20+ years XP
- Data-intensive systems – design and optimisation
- .Net, C#, LINQ, C++ STL, T-SQL

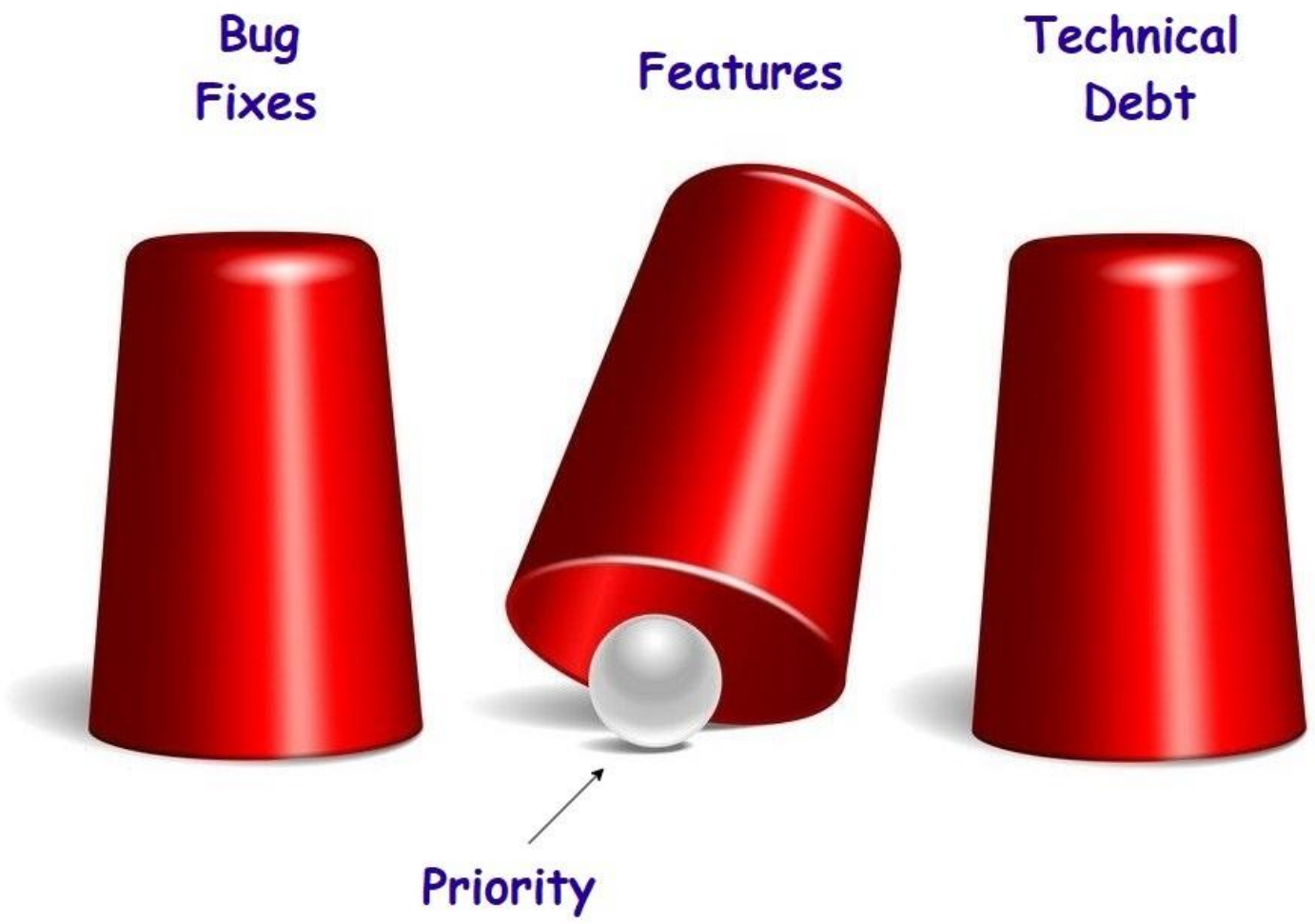
Get in Touch

- https://www.linkedin.com/in/marcinzelewski/
- marcin.zelewski@gmail.com

The Three Cups Game



Software Development



Definitions

Feature:

- Functionality or a behaviour that should be in the system,

  but isn't.

Bug:

- Functionality or a behaviour that is already in the system,

  but shouldn't.

Technical debt:

- Usually neither of the two above 😜

What is technical debt?

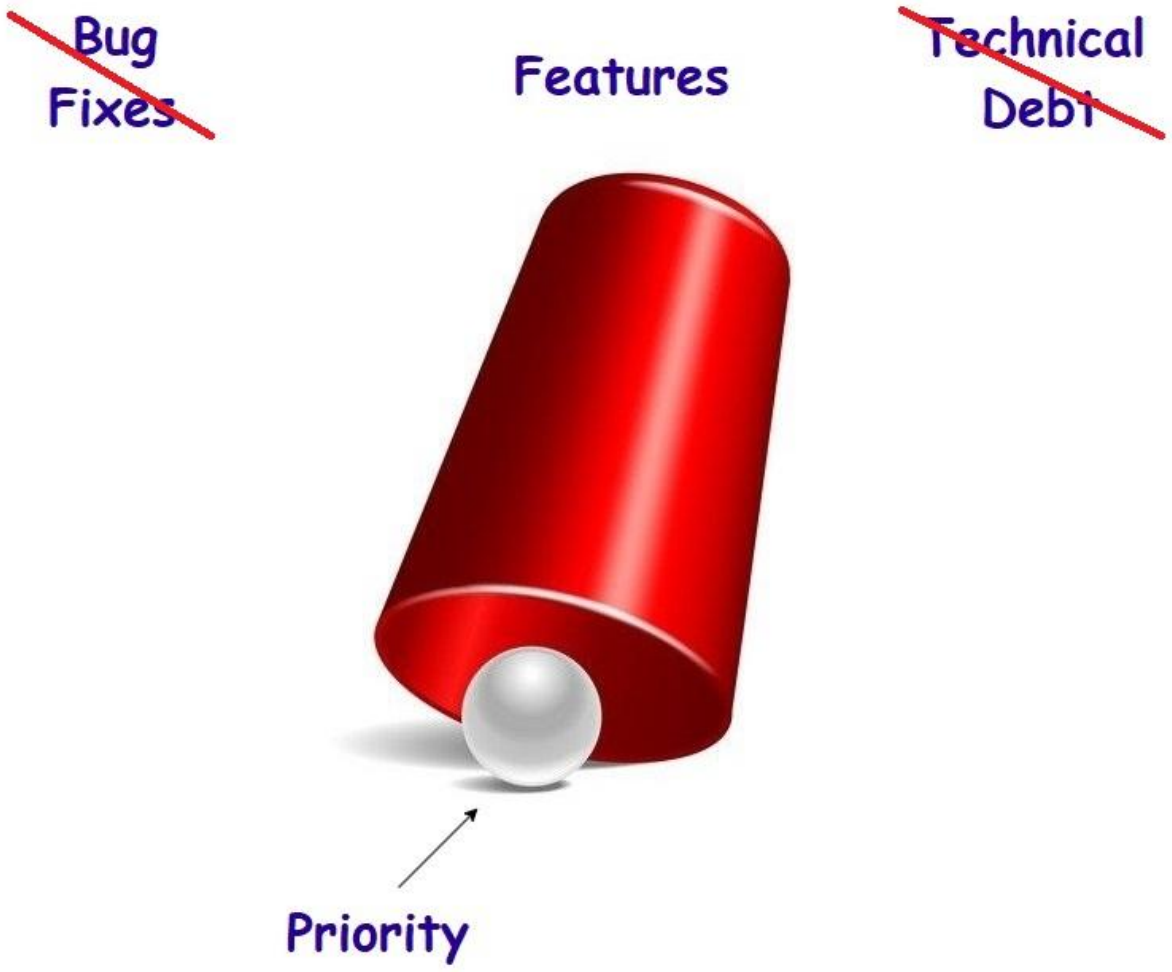
- insufficient technical documentation,
- insufficient coverage in automatic tests/manual test plans,
- temporary workarounds developed under time constraints,
- hard-coded settings, lack of flexibility or configuration,
- no coding standards, poor code reuse/redundancy, “dead code” -> refactoring
- inappropriate architecture, naive/brute-force algorithms,
- insufficient performance/scalability/resource management,
- missing tools/libraries that address the problem better.

When is technical debt not a debt?

- proof of concept and prototyping,
- low value, low impact projects and rarely used auxiliary tools,
- stable applications - no significant extensions are expected,
- deliberate strategic decision to prioritize speed of delivery over completeness and perfection (MVP scenario),

WHEN COST OF ADDRESSING THE DEBT OUTWEIGHS BENEFITS OF HAVING WELL DOCUMENTED, MAINTAINABLE, PERFORMANT, AND SCALABLE SOLUTION.

A winning scenario



Is it appropriate to look at the problem in this way?

The type of work item is not a priority criteria.

Stakeholder / Product Owner point of view:

- investment, piece of functionality, requirement, user story

Development / Cross Functional Team point of view:

- piece of work, task or collection of tasks

Risk based approach

Risks external to cross functional team:

Risk associated with **not delivering** the change – evaluate consequences (cost) and probability:

- Product strategy, roadmap
- Competitive advantage,
- Value to the business,
- Impact on users and their satisfaction,
- Reputation,
- Bugs and incidents: Severity, Frequency, Impact on teams.

Risk based approach

Risks external to cross functional team:

Actors / Stakeholders:

- Project Manager,
- Product Owner and Scrum Master,
- Sales and Marketing,
- Customer Support,
- Compliance,
- Quality Assurance,
- Other teams and departments

Risk based approach

Risks internal to cross functional team:

Risk associated with **delivering** the change that may introduce potential issues:

- Complexity,
- Cost to the business [effort estimation], Time To Market,
- Interaction with other work items or teams.

Actors:

- Development/Cross-Functional Teams
- Quality Assurance

Decision matrix

Dimensions:

- **Priority** – relates to risks external to the development team,
- **Complexity** – relates to risks internal to the development team

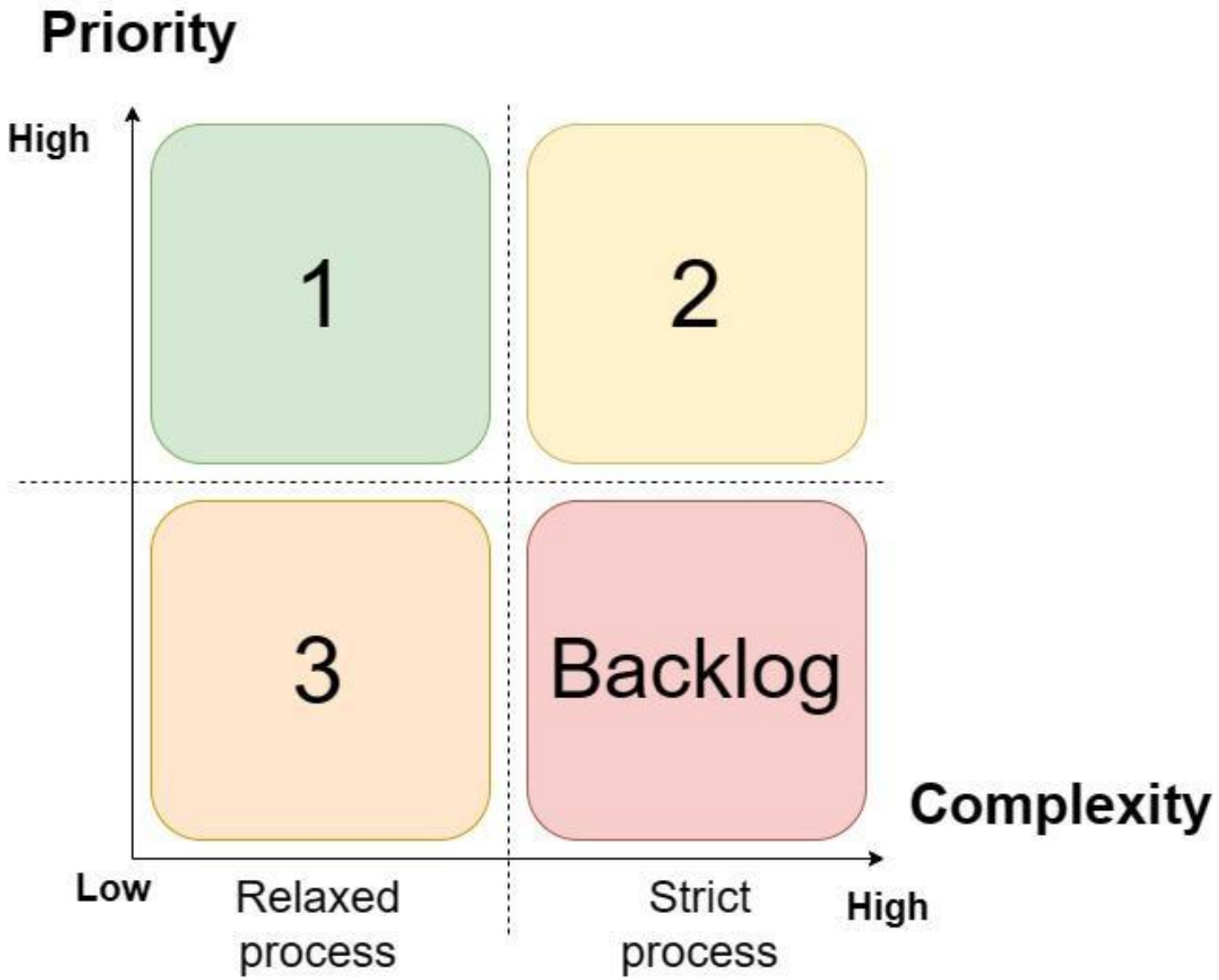
Values:

- **Low** / **High** [**NO** Medium, Average, Undecided, null, etc.]

Thresholds:

- Complexity: time to delivery (e.g. 1, 2, 3 working days)
- Priority: ???

Decision matrix



High Priority Items

1

Critical bugs and features with high value and low complexity and cost:

- address immediately outside of your sprint boundaries using maintenance time allocation,
- apply relaxed development process,

2

Urgent features and issues that align with project goals, but are time consuming or complex:

- plan and prioritize in your sprints as sprint objective items,
- apply strict development process,

Low Priority Items

3

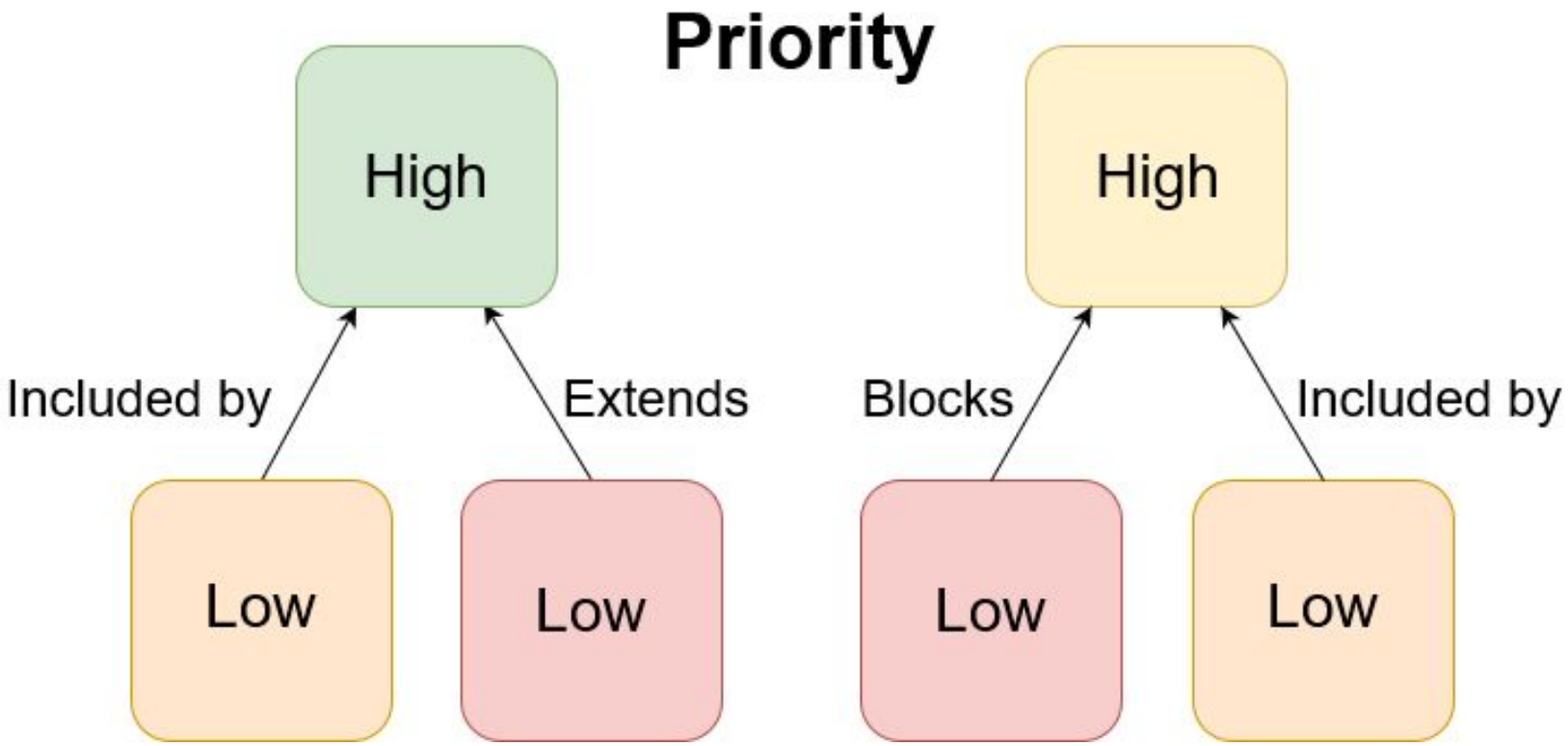
Non urgent features and issues with low cost:

- plan and prioritize in your sprints if time permits **OR** if the area of code is being worked on,
- apply relaxed development process,

Backlog

- DO NOT IMPLEMENT UNTIL VALUE IN EITHER OF DIMENSIONS IS REASSESSED
- do not remove items from the backlog as they can contain useful descriptions of decisions and determinations on priority and complexity

Dependencies between Items



WHEN A LOW PRIORITY ITEM IS INCLUDED IN, OR BLOCKS A HIGH PRIORITY ITEM - WE MUST RAISE PRIORITY TO HIGH

Process

Process	Strict	Relaxed
Requirement and design documentation	Created and maintained. Covering requirements, system architecture, database schemas, and interface designs	Created and/or updated if needed - in discretion of the team
Implementation	Follows coding standards and guidelines, code reviews	Emphasis on delivering working improvement quickly
Acceptance criteria	All agreed criteria must be met	Important criteria met and improvements made
Version control	Separate feature branch, PR	Change made on the trunk
Test coverage	Satisfactory coverage and quality of automated/manual tests	Expanding tests should be considered and done if necessary

When to reassess work items?

Several approaches, dependent on methodology:

- Ad-hoc review meetings,
- Review meetings at regular time intervals (each month, quarter),
- Start of each sprint,
- Start of a new development phase,
- When there are significant changes in the product strategy and business goals.

CONTINUOUSLY, AS SOON AS A NEW PIECE OF RELEVANT INFORMATION BECOMES AVAILABLE

Back to the beginning...

Marcin Zelewski, MSc. Eng. Computer Science

Freelance Software Developer, 20+ years XP

Get in Touch

- https://www.linkedin.com/in/marcinzelewski/
- marcin.zelewski@gmail.com